PLANS

Operator-Review Remediation Plans — nezha LLM provider keys + live LLMsVerifier wiring

Revision: 1 **Last modified:** 2026-06-16T14:28:56Z **Author:** background investigation agent (READ-ONLY, non-committing parallel stream) **Scope:** evidence-backed plans for operator to apply when back. NOTHING was applied. No secret VALUE is printed anywhere in this document (NAMES + lengths/states only, §11.4.10). **Discipline:** superpowers:systematic-debugging (root-cause-first) + §11.4.6 (no-guessing — every claim below is captured FACT) + §11.4.99 (latest-source research, cited).

ITEM 1 — `.env.nezha` line 48 malformed + 9 provider keys empty

Root cause (FACT — two distinct problems, not one)

The original hypothesis ("line 48 malformed → OPENAI/ANTHROPIC keys not injected") is **partially correct but the causal chain is different**. Systematic-debugging isolated TWO independent facts:

Fact 1A — line 48 is an unparseable statement (the warning source). Line 48 is a single physical line containing 31 space-separated bare provider-key NAMES, with exactly ONE trailing `=value` on the last token only:

```
OPENAI_API_KEY ANTHROPIC_API_KEY DEEPSEEK_API_KEY ...
LLMSVERIFIER_API_KEY SSH_WORKER_PASSWORD=<value>
```

python-dotenv **1.2.1** (the installed version, `/opt/homebrew/lib/python3.9/site-packages/dotenv/`) emits, reproduced verbatim:

```
python-dotenv could not parse statement starting at line 48
python-dotenv could not parse statement starting at line 48
```

When line 48 is parsed **in isolation**, dotenv extracts **ZERO** keys (`count: 0`). So line 48 is pure garbage — it does NOT itself set/override any key. It is a redundant/leftover line: every one of its 31 names ALSO has its own proper `NAME=value` line elsewhere in the file (lines 17–47). **Line 48 should simply be deleted.**

Fact 1B — 9 keys are EMPTY because their OWN-LINE values are blank (the actual injection failure). The reason OPENAI/ANTHROPIC/etc. are not usable is NOT line 48 — it is that their own-line values are genuinely empty (`KEY=` with nothing after). Measured own-line value lengths (chars after first `=`, value never printed):

Key (own-line #)	own-line value length	container-injected state
`OPENAI_API_KEY` (17)	**0**	EMPTY
`ANTHROPIC_API_KEY` (18)	**0**	EMPTY
`QWEN_API_KEY` (21)	**0**	EMPTY
`XAI_API_KEY` (27)	**0**	EMPTY
`TOGETHER_API_KEY` (29)	**0**	EMPTY
`NLP_CLOUD_API_KEY` (40)	**0**	EMPTY
`SARVAM_API_KEY` (42)	**0**	EMPTY
`LLMSVERIFIER_API_KEY` (46)	**0**	EMPTY
`SSH_WORKER_PASSWORD` (47)	**0**	EMPTY

The other 22 provider keys have nonempty own-line values and ARE injected SET into the running container (DEEPSEEK len35, ZHIPU len49, GEMINI len39, GROQ len56, CEREBRAS len52, KIMI len72, OPENROUTER len73, NVIDIA len70, etc.).

Sink-side evidence (§11.4.13 — `podman exec helixtranslate-monitor env`, NAMES + state only)

The 9 keys reporting EMPTY in the running container are EXACTLY the 9 keys with empty own-lines: `ANTHROPIC_API_KEY=EMPTY, LLMSVERIFIER_API_KEY=EMPTY, NLP_CLOUD_API_KEY=EMPTY, OPENAI_API_KEY=EMPTY, QWEN_API_KEY=EMPTY, SARVAM_API_KEY=EMPTY, SSH_WORKER_PASSWORD=EMPTY, TOGETHER_API_KEY=EMPTY, XAI_API_KEY=EMPTY`. All 22 others = SET(len=N). Local file analysis and container reality agree exactly.

Remediation plan (operator to apply — NOT applied)

1. **Delete line 48 entirely.** It is unparseable garbage and a duplicate of lines 17–47. Removing it eliminates the `could not parse statement starting at line 48` warning with zero functional loss (every name it lists already has its own line).
    - After deletion, re-verify: `python3 -c "from dotenv import dotenv_values; dotenv_values('.env.nezha')"` must emit **no** parse warning.

2. **Populate the 9 empty own-lines** so each is a proper standalone `NAME=value`. The canonical, dotenv-correct form is one key per line:

   ```
   OPENAI_API_KEY=<real-openai-key>
   ANTHROPIC_API_KEY=<real-anthropic-key>
   QWEN_API_KEY=<real-qwen-key>
   XAI_API_KEY=<real-xai-key>
   TOGETHER_API_KEY=<real-together-key>
   NLP_CLOUD_API_KEY=<real-nlpcloud-key>
   SARVAM_API_KEY=<real-sarvam-key>
   LLMSVERIFIER_API_KEY=<real-llmsverifier-key>      # required
   for ITEM 2
   SSH_WORKER_PASSWORD=<real-ssh-worker-password>
   ```

   - Values with spaces/special chars MUST be wrapped in double quotes (`KEY="value with spaces"`) — none of the 9 currently have values to inspect, so this is a forward rule.
   - Operator decides which of the 9 actually need real credentials (some may be intentionally blank). At minimum, `OPENAI_API_KEY` + `ANTHROPIC_API_KEY` are needed for those providers, and `LLMSVERIFIER_API_KEY` is needed for ITEM 2's authenticated path.

3. **Re-deploy** so the corrected env_file is re-read: the helixtranslate services use `env_file: ./.env.nezha` (confirmed in `compose.nezha.yml`), which is read only at container (re)creation — a running container will NOT pick up edits live.

4. **Validate (sink-side, NAMES only):**

   ```
   podman exec helixtranslate-monitor env | grep -E '^(OPENAI|
   ANTHROPIC|QWEN)_API_KEY=' | sed 's/=.*/=<set?>/'
   ```

   Each populated key should now report nonempty. (Use the redacting `${#val}` length pattern from this investigation; never print the value.)

Evidence index (ITEM 1)

- Parse error reproduced: python-dotenv 1.2.1, warning fired ×2 at line 48.
- Line 48 in isolation → 0 keys extracted (it sets nothing).
- 9 own-line value lengths = 0; 22 own-line value lengths > 0.
- Container env confirms 9 EMPTY / 22 SET, matching the file exactly.

ITEM 2 — enable the live nezha LLMsVerifier upstream

Topology (FACT — `podman network inspect`)

Network	Subnet	Members
`helixtranslate_translator-network`	**10.89.11.0/24** (gw .1)	helixtranslate-{postgres,redis,grpc,server,monitor,api}

Network	Subnet	Members
`llmsverifier_default`	**10.89.9.0/24** (gw .1)	llmsverifier_{postgres,redis,prometheus,grafana,llm-verifier}_1

`llmsverifier_llm-verifier_1`: IP **10.89.9.76**, network alias **`llm-verifier`**, port **8080/tcp published to host 127.0.0.1:8080 only** (`map[8080/tcp:[{127.0.0.1 8080}]]`).

Reachability gap — corrected by live probe (FACT, read-only `wget` from inside helixtranslate-monitor)

The "different network → unreachable" hypothesis is **only partly true**. Three probes from the helixtranslate container:

Target	Result
`http://localhost:8080/api/health` (current code default)	**Connection refused** — nothing listens on the helixtranslate container's own loopback
`http://llm-verifier:8080/api/health` (verifier DNS alias)	**bad address** — alias only resolves inside `llmsverifier_default`; helixtranslate is not joined
`http://10.89.9.76:8080/api/health` (verifier IP)	**HTTP 200 `{"status":"healthy","database":"connected",...}`** ✅

So IP-level routing between the two podman bridges ALREADY works on this host (default inter-bridge routing). The real gaps are: **(a)** helixtranslate has NO verifier endpoint configured at all, and **(b)** the convenient DNS alias `llm-verifier` does not resolve cross-network.

Confirmed: `LLMSVERIFIER_ENABLED` and `LLMSVERIFIER_API_URL` are **both unset** in the helixtranslate container.

How helixtranslate reads the verifier endpoint (FACT — code trace)

- `internal/verifier/client.go:35` → `baseURL: cfg.APIURL`; client hits `<APIURL>/api/health` and `<APIURL>/api/models`.
- `internal/config/config.go:339` → env var **`LLMSVERIFIER_API_URL`** overrides `c.LLMsVerifier.APIURL`.
- `internal/config/config.go:336` → env var **`LLMSVERIFIER_ENABLED=true`** enables it; validateLLMsVerifierConfig (config.go:402) requires APIURL non-empty when enabled.
- config.json key: `llms_verifier.api_url`, default **`http://localhost:8080`** (wrong for containerized deploy — that's the container's own loopback).

- `pkg/bridge/bridge.go:107-137` → when `LLMSVERIFIER_API_URL` is set, the bridge switches to **HTTP mode** against the running service; if set-but-unreachable it errors loudly (good — no silent fallback).

⚠ DEPENDENCY (FACT) — verifier currently has ZERO verified models

`http://10.89.9.76:8080/api/models` returns **`{"count":0,"models":[]}`** (both from inside the helixtranslate container and host-side). So even after wiring, `/api/v1/verified-models` will return empty until the verifier actually verifies models — and the verifier needs provider API keys to do so. **ITEM 1 must be fixed first**, then a verifier verification run must populate models, before ITEM 2's validation probe can return real data. This is the load-bearing ordering constraint.

Remediation plan (operator to apply — NOT applied)

Two layers: a **runtime** fix (immediate, non-persistent) and a **persistent** compose fix.

Wiring option A (RECOMMENDED) — join helixtranslate to the verifier network + use the DNS alias.

1. Live (no restart — `podman network connect` attaches a network to a *running* container immediately under podman v4+/netavark; it does NOT survive `podman rm`/recreate):

   ```
   podman network connect llmsverifier_default helixtranslate-server
   podman network connect llmsverifier_default helixtranslate-monitor
   podman network connect llmsverifier_default helixtranslate-api
   ```

   After connect, `llm-verifier:8080` resolves from those containers.

2. Persistent (survives recreate) — in `compose.nezha.yml`:

   - Declare the verifier network as external and attach the relevant services:

     ```
     networks:
       translator-network:
         driver: bridge
       llmsverifier_default:
         external: true        # created by the LLMsVerifier
             compose project
     ```

     and add `- llmsverifier_default` to the `networks:` list of `helixtranslate-server`, `-monitor`, `-api`.
   - Set the endpoint env (use the alias once joined):

     ```
     LLMSVERIFIER_ENABLED=true
     LLMSVERIFIER_API_URL=http://llm-verifier:8080
     ```

     (in `.env.nezha`, or a compose `environment:` block).

Wiring option B (simplest, works NOW without any network change) — point at the verifier IP, since inter-bridge routing already works.

- Set in `.env.nezha`:

  ```
  LLMSVERIFIER_ENABLED=true
  LLMSVERIFIER_API_URL=http://10.89.9.76:8080
  ```

- Trade-off: `10.89.9.76` is the verifier's current dynamic IP — it can change if the verifier container is recreated (§11.4.111 resolve-by-name-not-by-ordinal favours Option A's stable alias). Use Option B only as a stopgap; Option A is the durable choice.

> NOTE: `LLMSVERIFIER_API_URL` is read at container (re)creation via `env_file`; setting it in `.env.nezha` requires a re-deploy. Option A's `podman network connect` is the only live (no-recreate) part.

1. **Re-deploy** helixtranslate after the `.env.nezha` / compose edits so the new env + network attachment take effect persistently.

Validation probe (operator — proves real data end-to-end)

Order matters (ITEM 1 first → verifier populated → then this):

1. From inside the container, confirm the configured endpoint is healthy + has models:

   ```
   podman exec helixtranslate-server sh -c 'wget -qO- http://llm-verifier:8080/api/health'   # Option A
   podman exec helixtranslate-server sh -c 'wget -qO- http://llm-verifier:8080/api/models'    # expect count>0 after a verify run
   ```

2. Hit helixtranslate's own endpoint (named in CLAUDE.md):

   ```
   curl -s http://<helixtranslate-api-host:port>/api/v1/verified-models   # expect real model list, not []
   ```

   PASS = `/api/v1/verified-models` returns a non-empty, real model array sourced from the live verifier.

Evidence index (ITEM 2)

- `podman network ls` + `inspect`: two distinct subnets (10.89.11 vs 10.89.9), member lists captured.
- verifier: IP 10.89.9.76, alias `llm-verifier`, port published 127.0.0.1:8080 only.
- live probes from helixtranslate-monitor: localhost→refused, alias→bad address, IP→HTTP 200 healthy.
- `/api/models` → `{"count":0,"models":[]}` (verify-run dependency).
- code trace: `client.go:35` APIURL → `config.go:339` `LLMSVERIFIER_API_URL` env → config.json `llms_verifier.api_url` default `http://localhost:8080`; `bridge.go:107-137` HTTP-mode switch.

§11.4.99 research citations (latest-source, accessed 2026-06-16)

- podman-network-connect man page (synopsis `podman network connect [options] network container`; `--alias` adds DNS-resolvable network-scoped aliases): https://docs.podman.io/en/latest/markdown/podman-network-connect.1.html and https://docs.podman.io/en/v5.2.5/markdown/podman-network-connect.1.html
- netavark/aardvark multi-network DNS (a container connected to multiple networks resolves containers on each joined network; one known fqdn-resolution caveat across networks): https://github.com/containers/aardvark-dns/issues/403 ; Red Hat podman new network stack: https://www.redhat.com/en/blog/podman-new-network-stack ; podman networking guide: https://oneuptime.com/blog/post/2026-02-02-podman-networking-guide/view
- HONEST GAP (§11.4.6): the official man pages do NOT explicitly state runtime-live vs restart-persistence for `network connect`. The runtime-live behavior under podman v4+/netavark is established practice; the non-persistence across `podman rm`/recreate is why Option A also requires the compose `external: true` change. This distinction is flagged, not glossed.

Anti-bluff statement

Both plans are evidence-backed (captured parse errors, file-structure facts, live container probes, code traces, cited research). No fix was applied; no `.env.nezha` edit; no container mutation; no commit. No secret value appears in this document. Validation steps are specified for the operator to confirm real working state (§11.4.69 sink-side proof) after applying.